

U3_A6 Análisis de estructuras

Integrantes del equipo:

- Uriel
- Diego
- Hassan

Fecha de entrega: 22/10/2025

1. Estructura switch en C++

Sintaxis:

cpp

```
switch (expresión) {  
    case valor1:  
        // Bloque de código si expresión == valor1  
        break;  
    case valor2:  
        // Bloque de código si expresión == valor2  
        break;  
    // ... más casos ...  
    default:  
        // Bloque de código si no coincide ningún caso  
}
```

Explicación:

- La estructura switch evalúa una expresión y ejecuta diferentes bloques de código dependiendo del valor resultante
- Cada case representa un valor posible que puede tomar la expresión

- El break es crucial para salir del switch después de ejecutar un caso
- El caso default es opcional y se ejecuta cuando ningún caso coincide

Ejemplo práctico:

cpp

```
int dia = 3;

switch (dia) {
    case 1:
        cout << "Lunes";
        break;
    case 2:
        cout << "Martes";
        break;
    case 3:
        cout << "Miércoles"; // Este se ejecutará
        break;
    default:
        cout << "Día no válido";
}
```

2. Estructura do-while en C++

Sintaxis:

cpp

```
do {
    // Bloque de código a ejecutar
}
```

```
} while (condición);
```

Explicación:

- El bucle do-while garantiza que el bloque de código se ejecute **al menos una vez**
- La condición se evalúa **después** de ejecutar el bloque de código
- Si la condición es verdadera, el bloque se repite
- Es ideal para menús interactivos y validación de entrada de usuario

Ejemplo práctico:

```
cpp
int numero;
do {
    cout << "Ingrese un número positivo: ";
    cin >> numero;
} while (numero <= 0);
```

3. Vigencia del lenguaje de programación C++

Discusión:

Uriel: C++ sigue siendo completamente vigente en aplicaciones donde el rendimiento es crítico. Es el lenguaje preferido para:

- Motores de videojuegos (Unreal Engine)
- Sistemas operativos
- Navegadores web (Chrome, Firefox)
- Sistemas embebidos y de tiempo real
- Aplicaciones financieras de alta frecuencia

Diego: Si bien es vigente, su uso se ha especializado. Para desarrollo web y móvil existen alternativas más productivas. Lenguajes como Rust ofrecen ventajas en seguridad de memoria sin sacrificar rendimiento.

Hassan: La evolución constante de C++ (estándares C++17, C++20, C++23) demuestra su vigencia. La industria mantiene millones de líneas de código en C++ que serían muy costosas de reescribir.

Conclusión: C++ mantiene su vigencia en nichos de alto rendimiento y sistemas críticos, evolucionando para mantenerse competitivo.

4. Ventajas principales de Python

Discusión:

Uriel: La sintaxis clara y legible de Python reduce el tiempo de desarrollo. Es casi como escribir pseudocódigo ejecutable.

Diego: El ecosistema de librerías (PyPI) es abrumador. Desde IA con TensorFlow hasta web con Django, existe una librería para casi todo.

Hassan: Su naturaleza multipropósito lo hace ideal para:

- Ciencia de datos y machine learning
- Automatización y scripting
- Desarrollo web rápido
- Prototipado ágil

Ventajas principales:

1. **Rápido desarrollo** - Menor tiempo entre idea e implementación
 2. **Comunidad enorme** - Gran cantidad de recursos y soporte
 3. **Multidisciplinario** - Usado por científicos, ingenieros, artistas, etc.
 4. **Baterías incluidas** - Librería estándar muy completa
-

5. Utilidad de las funciones vs código en main

Discusión:

Uriel: Las funciones permiten dividir problemas complejos en partes manejables. Esto facilita el debugging y el mantenimiento.

Diego: Para programas pequeños, escribir todo en main puede ser aceptable. Pero en proyectos reales, esto genera "código espagueti" imposible de mantener.

Hassan: Las funciones promueven la **reutilización** y el **trabajo en equipo**. Cada desarrollador puede trabajar en funciones diferentes simultáneamente.

¿Por qué usar funciones?

- **Modularidad:** Divide y vencerás
- **Reutilización:** Escribe una vez, usa muchas veces
- **Legibilidad:** Código más organizado y comprensible
- **Testing:** Más fácil probar unidades individuales
- **Mantenimiento:** Cambios localizados sin afectar todo el programa

Conclusión:

Siempre es mejor construir funciones que escribir todo en main.

El main ideal actúa como coordinador, llamando a funciones específicas para cada tarea.